# Download Studio — Complete Technical and Docker Guide

This document explains how Download Studio works, what was added to Dockerize it, how the portable image archive was created, how to move it to another computer, and why the generated files are separate.

# 1. What the application does

Download Studio is a local web interface around `download_single.sh`.

From the browser, you can:

- paste a video or playlist URL;
- select an output directory;
- start, cancel, or retry a download;
- follow live progress, speed, ETA, and processing state;
- receive a browser notification when the job finishes;
- keep recent activity after the application restarts.

The final download directory contains media files, not separate subtitle or log files. Metadata and the thumbnail are embedded into the final media file. The application does keep its own small history database at `/data/history.json`; this is application state, not a log file placed beside downloaded videos.

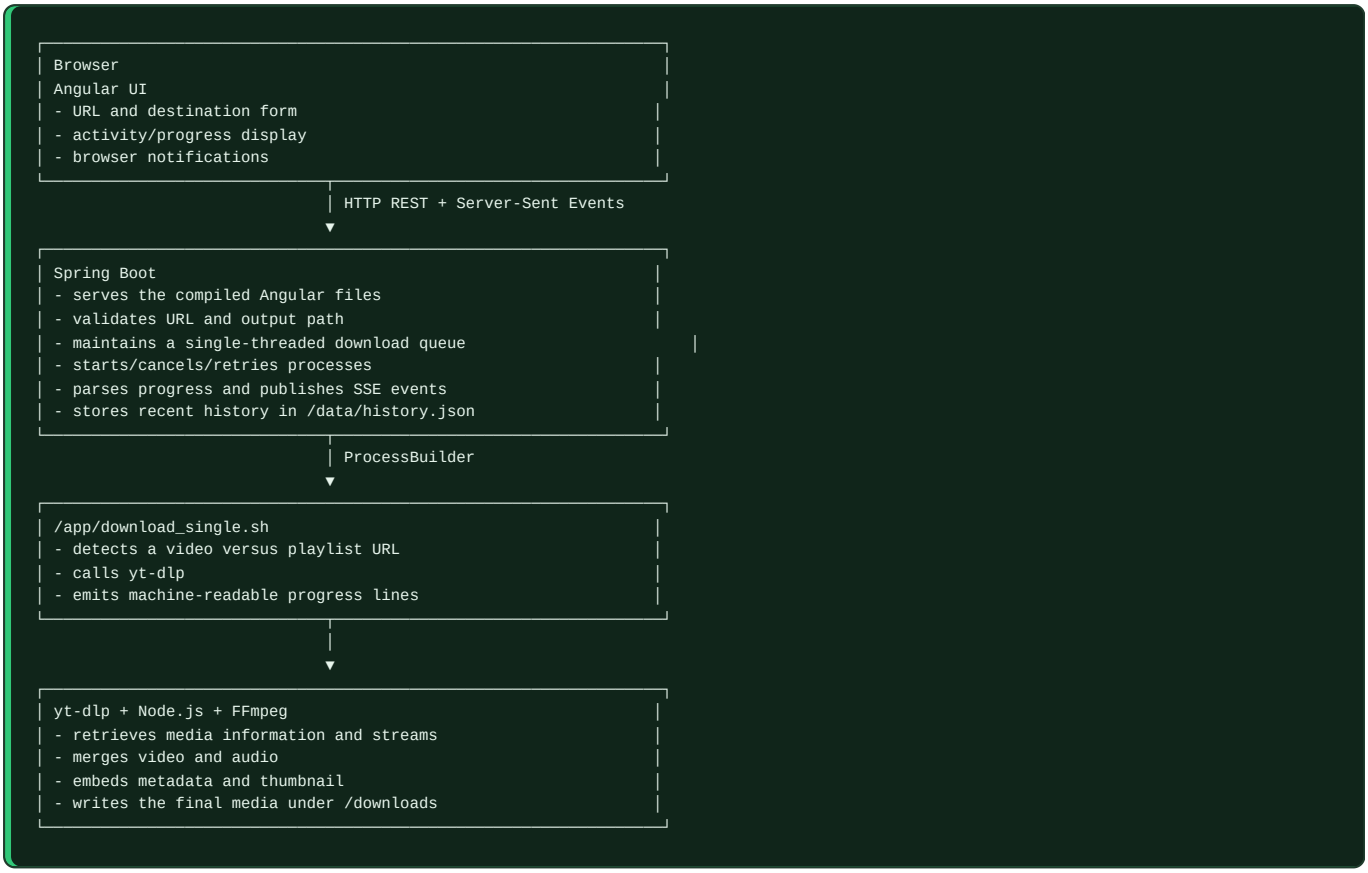
# 2. Why Angular alone was not enough

Angular runs inside the browser. A browser application cannot safely:

- execute `download_single.sh`;
- start `yt-dlp` or FFmpeg as operating-system processes;
- write freely to arbitrary filesystem directories;
- reliably keep downloads running after a page refresh;
- manage a persistent background queue.

Spring Boot supplies those operating-system capabilities. Angular is responsible for the interface, while Spring Boot validates requests, starts the script, parses progress, stores history, and sends live events back to Angular.

# 3. Architecture




### Request and progress flow

1. Angular sends `POST /api/downloads` with the URL, output directory, and optional browser name.
2. Spring Boot validates that the URL is HTTP/HTTPS and that the destination is an absolute writable path.
3. The job enters a single-thread queue, which prevents several large downloads from competing for disk and bandwidth simultaneously.
4. Spring Boot executes:

   ```
   bash /app/download_single.sh URL OUTPUT_DIRECTORY [BROWSER]
   ```

5. The shell script prints progress records such as `DL_PROGRESS`, `DL_PROCESSING`, and `DL_FILE`.
6. Spring parses those records into job state.
7. Spring sends job updates through `/api/downloads/events` using Server-Sent Events (SSE).
8. Angular updates the page without polling.
9. When a job changes to completed, failed, or canceled, Angular displays a browser notification if permission was granted.

### Important API endpoints

| Endpoint | Method | Purpose |
|---|---|---|
| `/` | GET | Compiled Angular interface |
| `/api/system/status` | GET | Checks `yt-dlp`, FFmpeg, script, and default directory |
| `/api/downloads` | GET | Lists recent jobs |
| `/api/downloads` | POST | Creates a job |
| `/api/downloads/{id}/cancel` | POST | Cancels a job |
| `/api/downloads/{id}/retry` | POST | Recreates a previous job |
| `/api/downloads/finished` | DELETE | Clears finished history |
| `/api/downloads/events` | GET | Live SSE job updates |
| `/api/directories` | GET/POST | Browses or creates output directories |

## 4. Project components

| Component | Location | Responsibility |
|---|---|---|
| Angular frontend | `download-studio/frontend/` | Interface, forms, live state, notification popup |
| Spring Boot backend | `download-studio/src/` | API, queue, process execution, history, SSE |
| Download script | `download_single.sh` | `yt-dlp` arguments and output naming |
| Docker image recipe | `download-studio/Dockerfile` | Builds and packages the complete application |
| Development Compose | `download-studio/docker-compose.yml` | Builds from source and runs locally |
| Portable Compose | `download-studio/docker-compose.portable.yml` | Runs an already-loaded image on another PC |
| Export helper | `download-studio/export-image.sh` | Builds, saves, and checksums the image |
| Build exclusions | `download-studio/Dockerfile.dockerignore` | Keeps dependencies, build output, docs, and archives out of Docker's build context |

### Organized repository layout

```
download-studio/
├── frontend/                    Angular source
├── src/                         Spring Boot source and tests
├── docs/                        manuals, PDF, and class diagrams
├── artifacts/                   portable Docker image and checksum
├── downloads/                   default host download directory
├── Dockerfile                   production image recipe
├── docker-compose.yml           local build/run configuration
├── docker-compose.portable.yml  source-free destination configuration
├── export-image.sh              portable image export helper
└── pom.xml                      Maven project
```

Only application source, build configuration, and deployment files remain at the project root. Generated manuals and large transport archives are isolated from the Docker build context.

# 5. What was changed for Docker

## Spring Boot configuration

The application settings support environment variables:

```
server.address=${SERVER_ADDRESS:127.0.0.1}
server.port=${PORT:8080}
download.script=${DOWNLOAD_SCRIPT:../download_single.sh}
download.history=${DOWNLOAD_HISTORY:${user.home}/.download-studio/history.json}
download.default-directory=${DOWNLOAD_DIRECTORY:${user.home}/Videos/YouTube}
```

Outside Docker, the secure default is `127.0.0.1`. Inside Docker, `SERVER_ADDRESS=0.0.0.0` is necessary so Docker can forward the host port into the container. Compose still publishes the host side only on `127.0.0.1`, so the website is not exposed to the local network.

Docker-specific paths are:

- downloader script: `/app/download_single.sh`;
- default download directory: `/downloads`;
- history file: `/data/history.json`;
- yt-dlp cache: `/data/cache`.

## Multi-stage Docker build

The Dockerfile has three stages.

**Stage 1: Angular build**

```
FROM node:24-bookworm-slim AS frontend-build
```

This stage installs exact frontend dependencies with `npm ci` and runs the Angular production build. The result is static HTML, CSS, and JavaScript under `frontend/dist`.

**Stage 2: Spring Boot build**

```
FROM maven:3.9.11-eclipse-temurin-17 AS backend-build
```

This stage copies the Angular build into Maven's resources and packages everything into `download-studio-1.0.0.jar`. The browser interface and backend therefore come from one Spring Boot process and one port.

**Stage 3: Runtime image**

```
FROM node:24-bookworm-slim AS runtime
```

The runtime installs only what the running application needs:

- Java 17 for Spring Boot;
- Node.js for yt-dlp's JavaScript runtime support;
- Python 3 for the installed yt-dlp launcher;
- yt-dlp for media downloads;
- FFmpeg for merging video/audio and embedding metadata;
- curl for the health check;
- tini for correct signal forwarding and child-process cleanup.

Maven, Angular CLI, source code, and frontend `node_modules` remain in build stages and are not copied into the final runtime image.

### Why the Docker build context is the parent folder

The command builds with `UsefulScripts` as its context because the application Dockerfile is under `download-studio/`, but `download_single.sh` is one directory above it.

```
docker build \
  --file download-studio/Dockerfile \
  --tag download-studio:1.0.0 \
  .
```

`Dockerfile.dockerignore` prevents large or irrelevant files such as `node_modules`, `target`, `dist`, downloaded videos, and existing `.tar` archives from being sent to Docker during the build.

## 6. Runtime filesystem and persistence

Containers have an isolated filesystem. A path such as `/home/your-user/Videos` on the host does not automatically exist inside the container.

Compose creates these mappings:

```
Host directory                   Container directory
-------------------------------------------------------
./downloads (or DOWNLOAD_DIR)  -> /downloads
Docker named volume            -> /data
temporary memory filesystem    -> /tmp
```

#### `/downloads`

This is a bind mount. Files written by the container appear in the chosen host directory and survive container replacement.

In the website's folder selector, select `/downloads`. Do not enter the host path, because the application sees paths from inside the container.

#### `/data`

This named Docker volume stores job history and yt-dlp cache. It survives `docker compose down` and container upgrades.

#### `/tmp`

The container root filesystem is read-only, so `/tmp` is supplied as a temporary in-memory filesystem. It is discarded when the container stops.

## 7. Security choices

The Compose configuration applies several restrictions:

- `USER node`: the application does not run as root;
- `read_only: true`: the container cannot modify its packaged application filesystem;
- `cap_drop: ALL`: Linux capabilities are removed;
- `no-new-privileges:true`: processes cannot gain additional privileges;
- `127.0.0.1:8080:8080`: the site is available only on the same computer by default;
- explicit `/downloads`, `/data`, and `/tmp` writable locations.

These settings limit damage if the web application or a downloaded media extractor behaves unexpectedly.

# 8. The two Compose files

There are two Compose files because the source computer and destination computer have different jobs.

### `docker-compose.yml`

Use this on the development/source computer. It contains a `build:` section, so Compose can rebuild the image from Angular, Java, and shell-script source.

```
docker compose up --build -d
```

### `docker-compose.portable.yml`

Use this on the destination computer after `docker load`. It contains only:

```
image: download-studio:1.0.0
```

It does not require or reference the source code. This is why you only need to copy the archive, checksum, and portable Compose file to the other computer.

They are not two applications and they do not create two different images. Both run the same image tag: `download-studio:1.0.0`.

# 9. Build and run from source

### Requirements

The host computer needs:

- Docker Engine;
- Docker Compose plugin;
- internet access during the first image build.

Java, Node.js, Maven, yt-dlp, and FFmpeg do not have to be installed on the host when using Docker; the image build provides them.

### Build and start

```
cd /home/houssamed/Documents/UsefulScripts/download-studio
mkdir -p downloads
docker compose up --build -d
```

Open:

```
http://127.0.0.1:8080
```

### Check status

```
docker compose ps
docker compose logs --tail=100 app
curl http://127.0.0.1:8080/api/system/status
```

Expected status response:

```
{
  "ready": true,
  "ytDlp": true,
  "ffmpeg": true,
  "script": true,
  "defaultDirectory": "/downloads",
  "message": "Ready to download"
}
```

### Stop or restart

```
docker compose stop
docker compose start
docker compose restart
```

### Stop and remove the container

```
docker compose down
```

This does not delete downloaded videos or the named history volume.

## 10. Choose a host download directory and port

Create the local environment file:

```
cd /home/houssamed/Documents/UsefulScripts/download-studio
cp .env.example .env
```

Edit `.env`:

```
DOWNLOAD_DIR=/home/your-user/Videos/YouTube
DOWNLOAD_STUDIO_PORT=8080
```

Then run:

```
docker compose up -d
```

The directory should exist and should be writable by the user running inside the image. On a normal Linux desktop where your user ID is `1000`, this usually works directly.

If port 8080 is already occupied:

```
DOWNLOAD_STUDIO_PORT=8090
```

Then open `http://127.0.0.1:8090`.

## 11. How the portable `.tar` was created

The helper script performs three operations.

### Step 1: build the image

```
docker build --pull \
  --file /home/houssamed/Documents/UsefulScripts/download-studio/Dockerfile \
  --tag download-studio:1.0.0 \
  /home/houssamed/Documents/UsefulScripts
```

`--pull` checks for newer base-image versions. `--tag` gives the image the stable name used by both Compose files.

### Step 2: save the image

```
docker save \
  --output artifacts/download-studio-1.0.0.tar \
  download-studio:1.0.0
```

`docker save` exports the local Docker image into a portable archive. The archive contains the image manifest, configuration, tags, and filesystem layers. It is not merely the JAR, and it is not a source-code archive.

Because all runtime dependencies are in those layers, the destination computer only needs Docker.

### Step 3: create the checksum

```
sha256sum artifacts/download-studio-1.0.0.tar \
  > artifacts/download-studio-1.0.0.tar.sha256
```

The complete automated command is:

```
cd /home/houssamed/Documents/UsefulScripts/download-studio
./export-image.sh
```

You may optionally specify another archive path:

```
./export-image.sh /path/to/download-studio-1.0.0.tar
```

## 12. Why there are a `.tar` and a `.sha256` file

These are not two Docker images.

#### `download-studio-1.0.0.tar`

This is the actual Docker image archive. It is approximately 345 MB and is required on the destination computer.

#### `download-studio-1.0.0.tar.sha256`

This is a tiny text file containing a cryptographic fingerprint of the large archive. Its purpose is to detect an incomplete or corrupted copy.

Current checksum:

```
e9217dfc09532357401a8124bb8ff7ed44b552dda8afbd81e2d1cf62a1228592
```

After copying both files, verify them with:

```
sha256sum --check download-studio-1.0.0.tar.sha256
```

Expected output:

```
download-studio-1.0.0.tar: OK
```

If it reports `FAILED`, do not load the archive; copy it again.

# 13. Move the application to another PC

## Files to copy

Copy these three project files into one directory on the destination computer:

```
artifacts/download-studio-1.0.0.tar
artifacts/download-studio-1.0.0.tar.sha256
docker-compose.portable.yml
```

The first two paths show their locations on the source computer. You may copy the files themselves into the same flat destination directory as the portable Compose file.

The destination computer does not need the Angular source, Java source, Maven files, or shell script separately. They are already inside the Docker image.

## Destination requirements

- Docker Engine;
- Docker Compose plugin;
- an AMD64/x86-64 processor for the currently exported image.

The current archive is `linux/amd64`. If the other computer is ARM64, such as some Raspberry Pi systems or Apple Silicon Linux environments, rebuild a compatible image or create a multi-platform image instead of using this archive directly.

## Verify, load, and start

Open a terminal in the directory containing the three copied files:

```
sha256sum --check download-studio-1.0.0.tar.sha256
docker load --input download-studio-1.0.0.tar
mkdir -p downloads
docker compose --file docker-compose.portable.yml up -d
```

Open:

```
http://127.0.0.1:8080
```

In the website, select `/downloads`. The actual videos will appear inside the `downloads` directory beside `docker-compose.portable.yml`.

## Use a different destination directory

Linux/macOS shell:

```
DOWNLOAD_DIR=/absolute/path/to/videos \
docker compose --file docker-compose.portable.yml up -d
```

Or create a `.env` file beside the portable Compose file:

```
DOWNLOAD_DIR=/absolute/path/to/videos
DOWNLOAD_STUDIO_PORT=8080
```

Then start normally:

```
docker compose --file docker-compose.portable.yml up -d
```

## 14. Image versus container versus archive

These terms represent different things:

| Term | Meaning |
|---|---|
| Source code | Angular, Java, and shell files used to build the application |
| Docker image | Read-only packaged application stored inside Docker |
| Docker container | A running instance created from the image |
| `.tar` archive | A transportable copy of the Docker image |
| Compose file | Instructions telling Docker how to run the image |
| Volume/bind mount | Storage that survives container removal |

Loading the archive creates the image in Docker:

```
docker load --input download-studio-1.0.0.tar
```

Starting Compose creates a container from that image:

```
docker compose --file docker-compose.portable.yml up -d
```

## 15. Video and playlist behavior

The script determines the layout from the URL:

- a normal video URL uses `/downloads/VIDEO_TITLE.EXT`;
- a URL containing `/playlist?` uses `/downloads/PLAYLIST_NAME/001 - VIDEO_TITLE.EXT`;
- a video URL with unrelated playlist parameters is forced to a single video;
- best video and best audio streams are selected and merged when necessary;
- MKV is the preferred merge container;
- metadata and thumbnail are embedded;
- no subtitle download options are enabled;
- no separate archive or `.log` file is written beside the video;

- `.part` files can exist temporarily while a download is active.

Diagnostic lines are still available through the web activity view and `docker compose logs`; those are runtime diagnostics, not downloaded files.

## 16. Browser notifications

The first download may cause the browser to request notification permission. Notifications appear only when:

- the browser supports the Notification API;
- notification permission is `Allow`;
- the UI receives the final job status over SSE;
- the page was already connected before the job changed to a finished state.

If notifications do not appear, check the permission for `http://127.0.0.1:8080` in the browser's site settings. The website also shows completion state inside the page even without operating-system notifications.

## 17. Public versus private videos

Public videos and playlists work directly in the container.

The original script can use `--cookies-from-browser`, but browser profiles and desktop keyrings normally live outside Docker. Selecting Chrome, Firefox, or another browser in the Dockerized UI will not automatically give the container access to that host browser profile.

Supporting private or age-restricted content would require an additional, carefully secured cookie-import design. Do not mount an entire browser profile into the container casually because it may expose login sessions.

## 18. Update and re-export the application

After changing Angular, Spring Boot, or `download_single.sh`:

```
cd /home/houssamed/Documents/UsefulScripts/download-studio
./export-image.sh
```

This rebuilds `download-studio:1.0.0`, overwrites the requested archive, and regenerates its checksum.

On the destination computer:

```
docker compose --file docker-compose.portable.yml down
docker load --input download-studio-1.0.0.tar
docker compose --file docker-compose.portable.yml up -d
```

Downloaded media and `/data` history remain outside the replaced container.

For clearer version management, change `1.0.0` consistently in the Dockerfile/export helper and both Compose files before distributing a materially new release.

# 19. Troubleshooting

### Port 8080 is already in use

```
DOWNLOAD_STUDIO_PORT=8090 \
docker compose --file docker-compose.portable.yml up -d
```

Open `http://127.0.0.1:8090`.

### Permission denied in `/downloads`

Confirm the host directory exists and is writable:

```
mkdir -p downloads
ls -ld downloads
```

On Linux, give your current user ownership of the chosen directory if appropriate:

```
sudo chown -R "$(id -u):$(id -g)" /absolute/path/to/downloads
```

Do not run a recursive ownership command against `/`, your entire home directory, or another broad path.

### Container is not healthy

```
docker compose --file docker-compose.portable.yml ps
docker compose --file docker-compose.portable.yml logs --tail=200 app
curl http://127.0.0.1:8080/api/system/status
```

### Check tools inside the image

```
docker run --rm --entrypoint sh download-studio:1.0.0 -c \
  'java -version; node --version; python3 --version; yt-dlp --version; ffmpeg -version | head -n 1'
```

### Check the installed image

```
docker image inspect download-studio:1.0.0 \
  --format 'ID={{.Id}} ARCH={{.Architecture}} OS={{.Os}} SIZE={{.Size}} USER={{.Config.User}}'
```

### View active containers

```
docker ps --filter name=download-studio
```

# 20. Chronological implementation record

This is the practical sequence of changes made to turn the original script into the current portable application.

1. The behavior and arguments of `download_single.sh` were reviewed first so the web application could preserve its video/playlist detection and output naming.

2. An Angular standalone application was created for the URL form, folder picker, queue cards, progress indicators, details panel, light/dark theme, transient messages, and browser notifications.
3. A Spring Boot service was created because browser-side Angular cannot execute shell processes or write directly to arbitrary directories.
4. REST endpoints were added for system readiness, directory browsing, creating folders, starting jobs, canceling, retrying, listing history, and clearing finished jobs.
5. A single-threaded executor was added so downloads run in order and do not unexpectedly saturate the machine with concurrent FFmpeg processes.
6. Structured progress templates were added to the shell script. Spring reads the process output and converts it into percentage, speed, ETA, title, processing state, and final-file information.
7. SSE was added so Angular receives immediate job updates without repeatedly polling the server.
8. The SSE disconnect failure was corrected. Closing or reloading a browser tab can throw `AsyncRequestNotUsableException` or `AsyncRequestTimeoutException`; those normal disconnects now receive no JSON error body, and dead emitters are removed. This avoids Spring trying to write a `Map` as `text/event-stream`.
9. In-page status messages were made transient: they automatically disappear after 4.2 seconds and also have a manual close button. Operating-system browser notifications are issued only for final job transitions.
10. Separate subtitle, download-archive, and persistent log-file behavior was removed from the downloader. Live diagnostic output remains in memory and history for the details view, while the media destination contains only final media and temporary `.part` files during an active transfer.
11. Spring configuration was parameterized with environment variables so script, history, port, bind address, and default output directory can change between native and Docker execution.
12. The Angular production build was embedded into the Spring Boot JAR, producing one deployable web service and avoiding a second runtime web server.
13. A three-stage Dockerfile was added to build Angular, build Spring Boot, and then create a smaller runtime-only image.
14. Java, Node.js, Python, yt-dlp, FFmpeg, curl, and tini were installed in the runtime image. A smoke test initially caught that `python3-minimal` lacked a standard module required by yt-dlp, so it was replaced with the complete `python3` runtime.
15. `/downloads` and `/data` persistence mappings were added, together with a writable temporary filesystem and a read-only application filesystem.
16. The container was hardened to run as the unprivileged `node` user with all Linux capabilities dropped and privilege escalation disabled.
17. A source-building Compose file and a source-free portable Compose file were created for their separate use cases.
18. An export helper was created around `docker build`, `docker save`, and `sha256sum`.
19. The finished image was started with the production security settings and tested through its actual HTTP and health endpoints.
20. The verified image was exported to the 345 MB archive supplied with this project.

## 21. Validation performed on the generated image

The final image was tested with the production-style restrictions from Compose.

Verified results:

- Docker image built successfully;
- architecture: `linux/amd64`;
- image runs as user `node`, not root;
- Java 17 starts correctly;
- Node.js starts correctly;
- Python 3 starts correctly;
- yt-dlp starts correctly;
- FFmpeg starts correctly;
- `/app/download_single.sh` exists and is executable;
- Spring Boot health endpoint reports `ready: true`;
- Angular homepage responds with HTTP 200;
- `/downloads` and `/data` mounts are writable;
- root filesystem remains read-only;

- Docker health status becomes `healthy`;
- both Compose files pass `docker compose config` validation;
- exported `.tar` passes its SHA-256 verification.

## 22. Quick command reference

### Build and run locally

```
cd /home/houssamed/Documents/UsefulScripts/download-studio
mkdir -p downloads
docker compose up --build -d
```

### Export for another PC

```
cd /home/houssamed/Documents/UsefulScripts/download-studio
./export-image.sh
```

### Load and run on another PC

```
sha256sum --check download-studio-1.0.0.tar.sha256
docker load --input download-studio-1.0.0.tar
mkdir -p downloads
docker compose --file docker-compose.portable.yml up -d
```

### Open the application

```
http://127.0.0.1:8080
```

### Stop it

```
docker compose --file docker-compose.portable.yml down
```